

Assignment 9: Distributed Databases using Replication

Eroniction Presley
CS 6650 — Scalable Distributed Systems
Khoury College of Computer Sciences, Northeastern University

April 2026

Repository: <https://github.com/Eroniction14/Scalable-Distributed-Systems>

Contents

1	How the Code Works	2
1.1	Overview	2
1.2	Core KV Store	2
1.3	Leader-Follower Architecture	2
1.4	Leaderless Architecture	3
1.5	The <code>local_read</code> Endpoint	3
1.6	Inter-Node Communication	3
2	N/R/W Implementation Details	3
2.1	W=5, R=1 (Strong Write, Fast Read)	3
2.2	W=1, R=5 (Fast Write, Heavy Read)	4
2.3	W=3, R=3 (Quorum)	4
2.4	Leaderless W=N, R=1	4
3	Unit Test Results	4
4	Load Test Results & Analysis	5
4.1	Read Latency	5
4.2	Write Latency	6
4.3	Stale Reads	7
4.4	Read-Write Interval Distribution	7
5	Which Configuration for Which Application?	8
6	How the Load Test Generator Works	9

1 How the Code Works

1.1 Overview

The system is an in-memory Key-Value store implemented in Go, deployed as a 5-node cluster using Docker Compose. Two database architectures are implemented: **Leader-Follower** and **Leaderless**, both sharing the same simple API (`/set` and `/get`) but differing in how writes are coordinated and replicated.

1.2 Core KV Store

Every node maintains a `KVStore` struct—a thread-safe hashmap (`sync.RWMutex + map[string]Entry`) where each entry holds a `Value` and a logical `Version` number. The version is critical for two things: (1) resolving which value is “newest” when reading from multiple nodes, and (2) detecting stale reads in the load tester.

When a node writes locally via `SetLocal()`, it increments the version atomically. When a node receives a replicated write via `Set()`, it only applies the update if the incoming version is strictly greater than the current version—this prevents out-of-order replication messages from overwriting newer data.

1.3 Leader-Follower Architecture

Write path: All client writes go to the Leader. The Leader:

1. Writes to its own local store (`SetLocal`) and gets a new version number.
2. Spawns goroutines to replicate to each Follower concurrently via `POST /replicate`.
3. Waits for enough Follower ACKs to satisfy the write quorum W . The Leader counts itself as 1, so it needs $W - 1$ Follower ACKs.
4. If enough ACKs arrive, returns `201-Created`. Otherwise returns `503`.

Read path: Depends on the read quorum R :

- **R=1:** The Leader just reads its own local store. Instant.
- **R=3 or R=5:** The Leader reads locally AND queries Followers via `GET /internal_read`. It collects responses from $R - 1$ Followers, compares all versions, and returns the entry with the highest version.

Quorum configuration is controlled entirely by environment variables W and R , passed through Docker Compose. No code changes are needed to switch between $W5R1$, $W1R5$, and $W3R3$.

Tricky parts:

- The delay simulation is important: the Leader sleeps 200ms after each Follower replication message, and Followers sleep 100ms before applying updates. These delays create a realistic inconsistency window.

- The quorum waiting logic uses a buffered channel (`ackCh`) to collect ACKs non-blockingly. It breaks out as soon as enough ACKs arrive rather than waiting for all Followers.

1.4 Leaderless Architecture

Write path: Any node can receive a write—it becomes the **Write Coordinator** for that request. The Coordinator:

1. Writes locally (`SetLocal`).
2. Replicates to ALL other 4 peers concurrently via `POST /replicate`.
3. Since $W = N$, it must wait for ALL peers to ACK before returning `201-Created`.

Read path: Since $R = 1$, each node just reads its own local store and returns immediately. No coordination needed for reads.

Key difference from Leader-Follower: There’s no fixed leader. The load tester sends writes to random nodes, so any node can be the coordinator for any write. This distributes write load evenly but means there’s no single “source of truth” node.

1.5 The `local_read` Endpoint

This is a testing-only endpoint on Follower/Leader nodes that reads directly from the local store without any quorum logic. During the replication window, `local_read` on a Follower will often return stale data because the Follower hasn’t received the update yet.

1.6 Inter-Node Communication

All inter-node communication uses simple HTTP/JSON. The `/replicate` endpoint handles write propagation, and `/internal_read` handles leader-initiated reads from Followers.

2 N/R/W Implementation Details

2.1 W=5, R=1 (Strong Write, Fast Read)

On write: Leader writes locally + sends to all 4 Followers + waits for ALL 4 ACKs. The write is slow ($\sim 105\text{ms}$) because it blocks until every node has the data.

On read: Leader returns its own value. Since $W = 5$ guarantees all nodes are up-to-date when the write completes, reading from any single node is safe.

Consistency guarantee: Very strong. Stale reads are nearly impossible after a write is acknowledged.

2.2 W=1, R=5 (Fast Write, Heavy Read)

On write: Leader writes only to its local store and immediately returns. Replication to Followers happens asynchronously in the background. Writes are extremely fast ($\sim 2\text{ms}$).

On read: Leader reads from itself AND all 4 Followers, then picks the highest version. This compensates for potentially stale Followers by always checking everyone.

Consistency guarantee: Reads are eventually consistent— $R = 5$ reads all nodes and picks the latest version, but during the replication window, stale data can be returned because the version tracker detects that the write was acknowledged (client got 201) before all followers were updated.

2.3 W=3, R=3 (Quorum)

On write: Leader writes locally + waits for 2 Follower ACKs (total $W = 3$). Faster than $W = 5$ because it only needs a majority.

On read: Leader reads locally + queries 2 Followers (total $R = 3$). Picks the highest version.

Consistency guarantee: Since $W + R = 6 > N = 5$, there's guaranteed overlap—at least one node in every read quorum participated in the write quorum. This ensures the read will find the latest version. A nice middle ground between the two extremes.

2.4 Leaderless W=N, R=1

On write: The coordinator writes to itself + all 4 peers. Must wait for all to ACK. Writes are the slowest ($\sim 305\text{ms}$) because the coordinator must reach every single node.

On read: Node returns its own local value. Since $W = N$ ensures all nodes are updated before the write completes, $R = 1$ is safe.

3 Unit Test Results

All 6 unit tests pass, validating both consistency guarantees and the existence of inconsistency windows.

Test	Result	What it proves
Leader read after write	PASS	Leader returns consistent data after write ACK
Follower read after ACK	PASS	All 4 Followers consistent after Leader ACK
Local read inconsistency	PASS	<code>local_read</code> catches stale data during replication
Leaderless inconsistency window	PASS	Non-coordinator nodes return stale data during replication
Coordinator read after ACK	PASS	Coordinator's data is consistent after write completes
Other node read after ACK	PASS	Other nodes consistent after $W=N$ replication finishes

Table 1: Unit test results for both Leader-Follower and Leaderless databases.

4 Load Test Results & Analysis

Load tests were run with 2000 requests, 20 concurrent workers, and 20 unique keys across all four write ratios (1%, 10%, 50%, 90%) for each of the four database configurations.

4.1 Read Latency

Config	1% Writes (p50/p99)	10% Writes	50% Writes	90% Writes
W5R1	51.8 / 93.1 ms	51.3 / 74.7 ms	51.7 / 72.5 ms	52.0 / 70.2 ms
W1R5	58.0 / 90.9 ms	57.3 / 93.5 ms	57.1 / 79.6 ms	68.2 / 131.5 ms
W3R3	53.8 / 91.2 ms	51.6 / 65.0 ms	52.2 / 67.6 ms	52.1 / 75.3 ms
Leaderless	14.3 / 40.9 ms	0.8 / 21.7 ms	1.1 / 27.9 ms	2.0 / 19.2 ms

Table 2: Read latency (p50 / p99) across all configurations and write ratios.

Analysis: Leaderless reads are dramatically faster (sub-2ms at p50) because $R = 1$ means the node just returns from its local hashmap—no network round trips, no follower sleep delays. Among leader-follower configs, W1R5 has the highest read latency because every read must query all 5 nodes and wait for all responses; the p99 balloons to 131ms at 90% writes because heavy concurrent writes create contention on the Followers. W5R1 and W3R3 have similar read latencies (~ 52 ms p50) because the dominant factor is the 50ms Follower sleep on read requests.

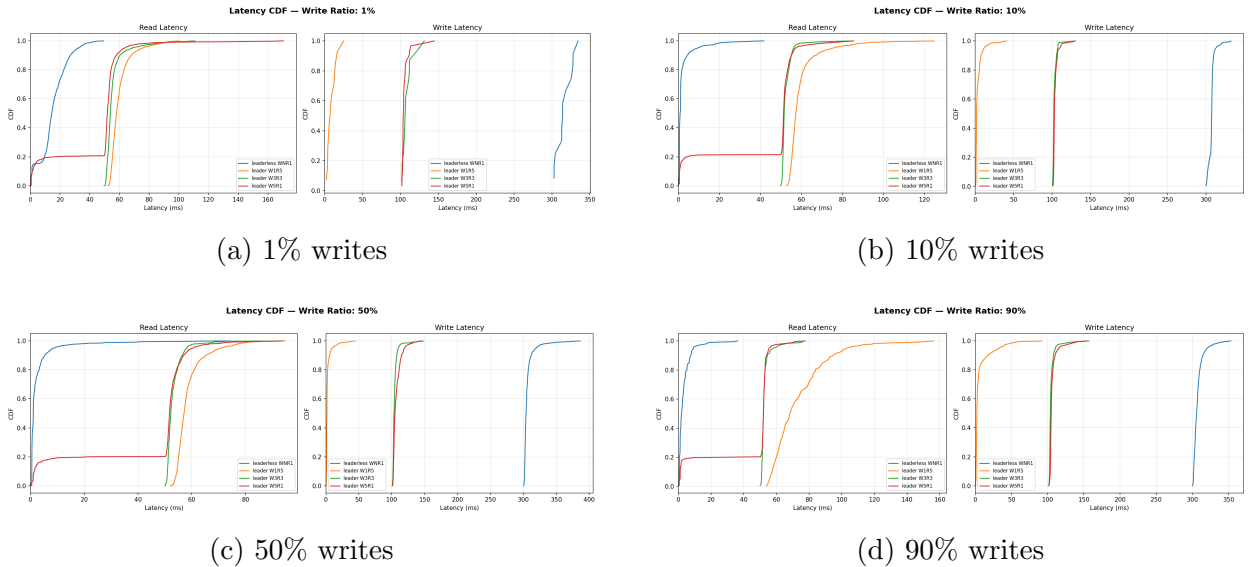


Figure 1: Latency CDF for reads and writes across all four write ratios.

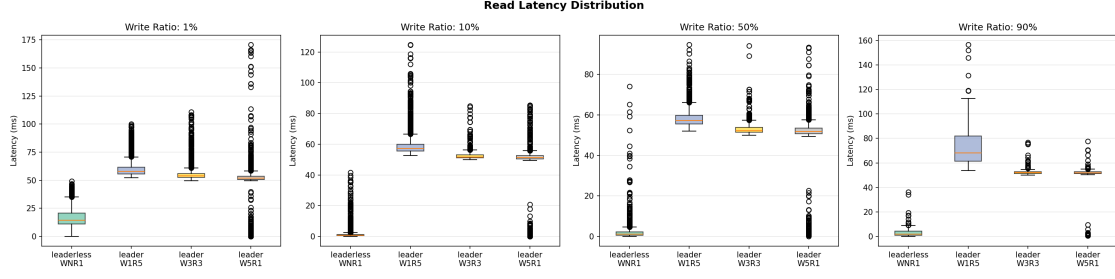


Figure 2: Read latency box plots showing distribution and long-tail outliers.

4.2 Write Latency

Config	1% Writes (p50/p99)	10% Writes	50% Writes	90% Writes
W5R1	103.6 / 113.7 ms	102.9 / 120.6 ms	105.0 / 133.5 ms	104.9 / 137.6 ms
W1R5	7.1 / 16.6 ms	1.8 / 20.4 ms	1.6 / 26.2 ms	2.2 / 55.0 ms
W3R3	106.0 / 122.7 ms	102.1 / 109.0 ms	103.5 / 129.7 ms	103.7 / 129.7 ms
Leaderless	314.1 / 328.1 ms	307.5 / 321.4 ms	304.1 / 348.8 ms	305.8 / 338.6 ms

Table 3: Write latency (p50 / p99) across all configurations and write ratios.

Analysis: W1R5 has by far the fastest writes (~ 2 ms) because the Leader only commits locally. W5R1 and W3R3 have similar write latencies (~ 103 ms)—even though W3R3 only needs 2 Follower ACKs vs 4 for W5R1, the 200ms Leader sleep + 100ms Follower sleep creates a baseline that dominates. The leaderless database has the slowest writes (~ 305 ms) because the coordinator must replicate to all 4 peers, each incurring the 200ms + 100ms delay overhead.

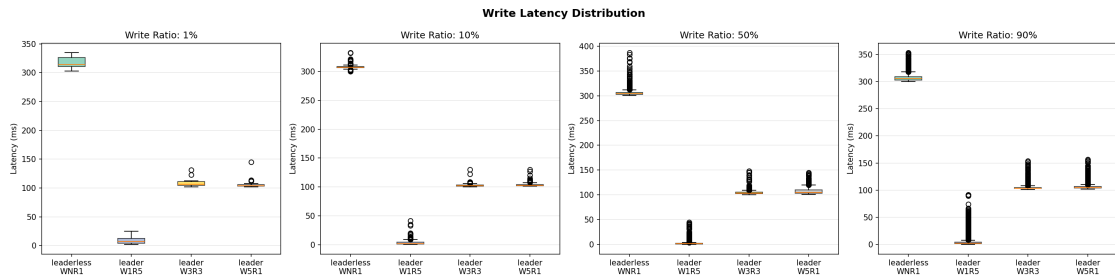


Figure 3: Write latency box plots showing distribution and long-tail outliers.

4.3 Stale Reads

Config	1% Writes	10% Writes	50% Writes	90% Writes
W5R1	0.00%	0.06%	0.00%	0.00%
W1R5	0.05%	1.84%	10.78%	19.03%
W3R3	0.00%	0.06%	0.21%	1.13%
Leaderless	0.00%	0.00%	0.00%	0.00%

Table 4: Percentage of stale reads detected by the load tester.

Analysis: This is the most revealing table. W1R5 shows a dramatic increase in stale reads as the write ratio increases—at 90% writes, nearly 1 in 5 reads is stale. This happens because writes return before replication completes, so Followers are frequently out of date. With 90% writes, the replication pipeline is constantly behind, and even though $R = 5$ reads all nodes and picks the max version, the version tracker in the load tester detects staleness because it knows the write completed (the client got 201) but the subsequent read returns a lower version.

W5R1 and Leaderless both show effectively 0% stale reads because $W = N$ (or $W = 5$) ensures all nodes are fully updated before the write returns. W3R3 shows a tiny amount of staleness (up to 1.13%) because the quorum overlap isn’t perfect in practice—it’s possible for a read to arrive at a node that is still in the process of being updated, even though the quorum math guarantees overlap in the steady state.

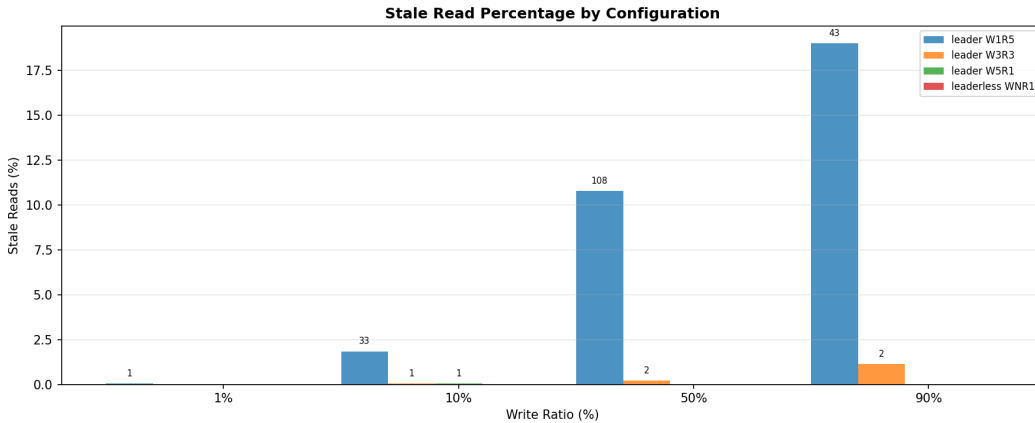


Figure 4: Stale read percentage by configuration and write ratio.

4.4 Read-Write Interval Distribution

The read-write interval measures the time between the most recent write to a key and a subsequent read of that same key.

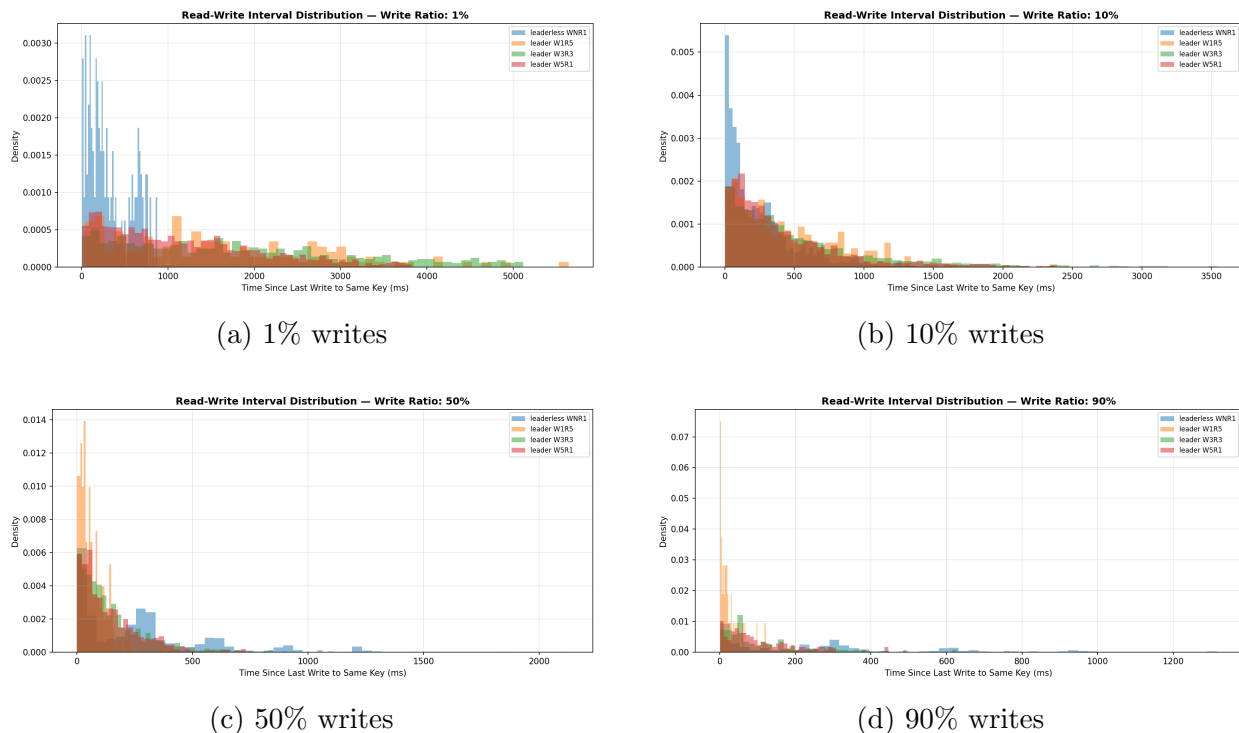


Figure 5: Distribution of time intervals between writing and reading the same key.

Analysis: At low write ratios (1%), intervals are large (p50 $\sim$ 1–1.5 seconds) because writes are infrequent. At high write ratios (90%), intervals shrink dramatically (p50 $\sim$ 15–80ms), confirming that our small key space (20 keys) successfully creates temporal locality. This temporal clustering is what makes stale read detection possible—if reads and writes to the same key were separated by seconds, replication would always complete in time and we’d never observe staleness.

5 Which Configuration for Which Application?

W=5, R=1 — Best for read-heavy, consistency-critical applications. Examples: configuration stores, DNS-like services, financial reference data. Writes are expensive but reads are fast and always consistent. If your application writes once and reads thousands of times, this is ideal.

W=1, R=5 — Best for write-heavy applications that tolerate eventual consistency. Examples: logging systems, event ingestion, sensor data collection. Writes are nearly instant, but reads are slow and may return stale data. Only suitable when you can tolerate reading slightly outdated values.

W=3, R=3 (Quorum) — Best general-purpose configuration. Examples: e-commerce inventory, user profile stores, session management. Balances write speed with read consistency. The quorum overlap guarantees that reads will find the latest data in most cases, with only marginal staleness under extreme write loads.

Leaderless W=N, R=1 — Best for high-availability, read-dominated work-

loads. Examples: CDN edge caches, distributed configuration stores. Reads are the fastest of any configuration (sub-2ms). No single point of failure since any node can coordinate writes. However, writes are the slowest because every single node must acknowledge. Best when availability and read speed matter more than write throughput.

6 How the Load Test Generator Works

The load tester uses a **small key space** approach (default: 20 unique keys) to guarantee temporal locality. With 20 concurrent workers all reading and writing from only 20 keys, collisions are frequent—workers naturally end up reading a key shortly after another worker wrote to it.

For each request, the tester:

1. Randomly picks a key from **key-0** to **key-19**.
2. Decides read vs write based on the configured write percentage.
3. For writes: sends to the Leader (or random node in leaderless mode) and records the returned version in a **VersionTracker**.
4. For reads: sends to a random node and compares the returned version against the tracker's latest known version. If lower, it's flagged as stale.
5. Records latency, staleness, and the time since the last write to that key.

The small key space is the key design decision—with only 20 keys and 20 workers, the probability of a read hitting a recently-written key is high, which is exactly what we need to trigger and detect stale reads within the inconsistency window.